

Competitive Programming

Section 53: Advanced Combinatorics

Competitive Programming Course

2026-07-10

Table of contents

1	Counting structures	3
1.1	Catalan numbers	3
1.2	Derangements	3
1.3	Burnside's lemma	3
1.4	Recognition checklist	4
1.5	Catalan worked example	4
1.6	Burnside worked example	4
1.7	Modular implementation notes	5
2	Exercises	6
3	Derivations to reproduce under pressure	7

1 Counting structures

Advanced combinatorics is mostly about recognizing a known structure and checking its assumptions. Catalan counts balanced recursive objects, derangements count permutations with forbidden fixed points, and Burnside's lemma counts orbits under symmetry.

1.1 Catalan numbers

Catalan numbers count many recursively balanced structures:

- valid parenthesis strings;
- rooted binary tree shapes;
- triangulations of a convex polygon;
- monotone paths that do not cross a diagonal.

Formula:

$$\text{Cat}(n) = C(2n, n) / (n + 1)$$

Under prime modulus MOD, compute division by multiplying with modular inverse:

$$\text{Cat}(n) = C(2n, n) * \text{inv}(n+1) \bmod \text{MOD}$$

1.2 Derangements

A derangement is a permutation with no fixed point. Let $D[n]$ be the number of derangements of n elements.

Recurrence:

$$\begin{aligned} D[0] &= 1 \\ D[1] &= 0 \\ D[n] &= (n-1) * (D[n-1] + D[n-2]) \end{aligned}$$

Reason: element 1 goes to some j . Either j goes back to 1, leaving a derangement of $n-2$ elements, or j does not go back, which contracts into a derangement-like arrangement of $n-1$ elements.

1.3 Burnside's lemma

When objects are considered equivalent under symmetry, count orbits by:

number of orbits = average number of objects fixed by each symmetry

For necklaces of length n with k colors under rotation, a rotation by r positions has:

$\gcd(n, r)$

cycles, so it fixes:

$k^{\gcd(n,r)}$

colorings. Therefore:

$$\text{answer} = (1/n) * \sum_{r=0}^{n-1} k^{\gcd(n,r)}$$

Under a prime modulus, divide by n using $\text{inv}(n)$.

1.4 Recognition checklist

- “balanced recursive object” suggests Catalan.
- “permutation with forbidden fixed positions” suggests derangements or inclusion-exclusion.
- “same up to rotation/reflection” suggests Burnside or Polya.
- Always identify the symmetry group before applying Burnside.

1.5 Catalan worked example

For $n = 3$, valid parenthesis strings are:

((()))
(()())
(())()
()(())
()()()

So $\text{Cat}(3)=5$. Formula:

$$C(6,3)/(3+1) = 20/4 = 5$$

Under modulo arithmetic, compute $20 * \text{inv}(4) \bmod \text{MOD}$.

1.6 Burnside worked example

Necklaces of length 3 with 2 colors, up to rotation.

Rotations:

```
rotate by 0: gcd(3,0)=3 cycles -> 2^3 fixed colorings
rotate by 1: gcd(3,1)=1 cycle -> 2^1 fixed colorings
rotate by 2: gcd(3,2)=1 cycle -> 2^1 fixed colorings
```

Average:

$$(8 + 2 + 2) / 3 = 4$$

The four necklaces are represented by:

```
000, 111, 001, 011
```

where rotations of 001 are considered the same.

1.7 Modular implementation notes

For all three exercises, precompute enough factorials or powers before answering queries.

Catalan:

```
cat[n] = C(2n,n) * inv(n+1)
```

Derangements:

```
D[0]=1
D[1]=0
D[n]=(n-1) (D[n-1]+D[n-2])
```

Burnside:

```
ans = sum(pow(k, gcd(n,r), MOD) for r in 0..n-1) * inv(n)
```

The danger in Burnside is using the number of fixed positions instead of the number of cycles. A rotation fixes a coloring when all positions in each cycle share the same color.

2 Exercises

- A. Catalan Queries: Print Catalan numbers modulo $1e9+7$.
- B. Derangement Queries: Print the number of derangements of n modulo $1e9+7$.
- C. Necklace Orbits: Count color necklaces of length n with k colors up to rotation. #
Contest checklist

3 Derivations to reproduce under pressure

Catalan. $C[n]$ counts balanced parenthesis strings, binary-tree shapes, and noncrossing pairings. A first pair splits the inside into i pairs and the outside into $n-1-i$, giving $C[n] = \sum C[i] * C[n-1-i]$; $C[0]=1$. Trace: $C[1]=1, C[2]=2, C[3]=5$. For many queries precompute once; do not divide by $n+1$ modulo MOD unless its inverse is legal.

Derangements. Consider where element one goes. If it swaps with its image, there are $D[n-2]$ completions; otherwise merge the two labels and get $D[n-1]$. Hence $D[n] = (n-1) (D[n-1] + D[n-2])$, with $D[0]=1, D[1]=0$.

Burnside. A necklace orbit is an equivalence class under rotations. A rotation by r fixes $k^{\gcd(n,r)}$ colorings; average these fixed counts over all n rotations. For length three with two colors: $(2^3 + 2^1 + 2^1)/3 = 4$. The division uses $\text{inverse}(n)$ only under the local prime modulus and n not divisible by it.

Use `long long` in C++ and `pow(base,exp,MOD)` in Python. Test zero, one, a prime necklace length, and a composite length. Run `CP_TARGET=solution python3 sections/53_advanced_combinatorics` for small n , enumerate parenthesis strings, permutations, or colorings as an oracle.

State the invariant, choose the model, prove the operation, then code the smallest reliable version.